Lecture 25.07: Types of Naive Bayes Classifiers

📌 Topic Overview

In this lecture, Shraddha ma'am explores the three primary variants of Naive Bayes in Scikit-Learn: **GaussianNB**, **MultinomialNB**, and **BernoulliNB**, explaining when each should be applied based on the underlying data distribution.

🏛️ The Three Variants

1. Gaussian Naive Bayes (`GaussianNB`)

- **Data Type**: Continuous numerical variables (e.g., Age, Height, Temperature, Salary).
- **Assumption**: Features within each class follow a **Gaussian (Normal) Bell Curve**.
- Computes mean μ_k and variance σ_k^2 for each feature.

2. Multinomial Naive Bayes (`MultinomialNB`)

- **Data Type**: Discrete count data (e.g., word occurrence counts in text documents).
- **Typical Use Case**: Natural Language Processing (NLP), document classification, spam filtering with word frequencies.

3. Bernoulli Naive Bayes (`BernoulliNB`)

- **Data Type**: Binary boolean features (0 or 1).
- **Typical Use Case**: Short text classification where only word presence or absence matters (word present = 1, absent = 0).

📊 Selection Guide Flowchart



⚖️ Comparison Table

Variant	Input Data Type	Underlying Distribution	Primary Domain
GaussianNB	Continuous ($\mathbb{R}$)	Normal / Gaussian Distribution	Tabular sensor data, scientific metrics
MultinomialNB	Non-negative integers ($\mathbb{N}_0$)	Multinomial Distribution	NLP, text document categorization
BernoulliNB	Binary boolean ($[0, 1]$)	Bernoulli Distribution	Spam detection, sentiment on short tweets

---

💻 Python Implementation

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB

def select_naive_bayes_model(feature_type):
    """
    Selects and initializes the appropriate Naive Bayes variant:
    - 'continuous' -> GaussianNB (Bell curve, mean & variance)
    - 'counts'     -> MultinomialNB (Word frequencies, discrete counts)
    - 'binary'     -> BernoulliNB (Presence/absence 0 or 1)
    """
    if feature_type == 'continuous':
        model = GaussianNB()
        desc = "GaussianNB: Continuous real-valued features following Normal Distr
    elif feature_type == 'counts':
        model = MultinomialNB()
        desc = "MultinomialNB: Discrete integer count data (e.g., Bag-of-Words in
    elif feature_type == 'binary':
        model = BernoulliNB()
        desc = "BernoulliNB: Binary boolean features (e.g., word present=1, absent
    else:
        raise ValueError(f"Unknown feature type: {feature_type}")

    return model, desc

# --- Function Call & Testing ---
for f_type in ['continuous', 'counts', 'binary']:
    clf, description = select_naive_bayes_model(f_type)
    print(f"[{f_type.upper()}] => {clf.__class__.__name__}")
    print(f"  Usage: {description}")
```

🔑 Key Takeaways

1. Matching the correct Naive Bayes algorithm to your feature distribution is critical for accurate predictions.
2. For real-valued numbers, use `GaussianNB`.
3. For NLP word counts, use `MultinomialNB`.
4. For binary indicators, use `BernoulliNB`.